COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	10-01-2026	**DATE SUBMITTED:**	10-01-2026
NAME:	Andrew S. Doloso Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10
GRAPHS

1. PROCEDURE OUTPUT

ILO A.

main.cpp

Welcome | main.cpp | graph.h | bp1.cpp

Graph > main.cpp > main()

```cpp
#include <iostream>
#include "graph.h"

// graph implementation
int main() {
    // graph edges array
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list " << std::endl
              << "(start_vertex, end_vertex, weight):" << std::endl;

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

graph.h

Welcome | main.cpp | graph.h | bp1.cpp

Graph > graph.h > DiaGraph > DiaGraph(graphEdge [], int, int)

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>

// Stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// Structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // Insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph

public:
    adjNode **head; // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // Allocate new node array
        head = new adjNode*[N]();
        this->N = N;

        // Initialize head pointer for all vertices
        for (int i = 0; i < N; ++i) {
            head[i] = nullptr;
        }

```

```cpp
#ifndef GRAPH_H
class DiaGraph {
public:
    DiaGraph(graphEdge edges[], int n, int N) {
        for (int i = 0; i < N; ++i) {
        }

        // Construct directed graph by adding edges to it
        for (unsigned int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // Insert at the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            // Point head pointer to new node
            head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* current = head[i];
            while (current != nullptr) {
                adjNode* next = current->next;
                delete current;
                current = next;
            }
        }
        delete[] head;
    }
};

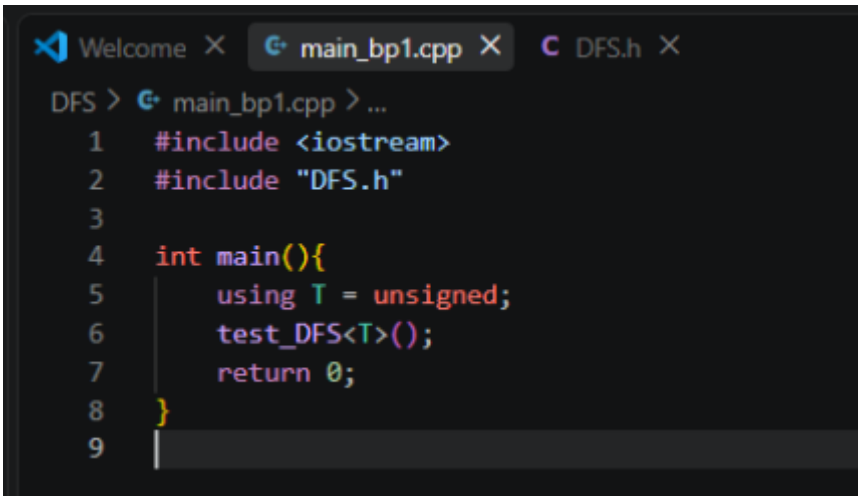
// Print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

#endif
```

OUTPUT

```
PS C:\Users\TIPQC\CPE010-doloso\B1> cd "c:\Users\TIPQC\CPE010-doloso\Graph\" ; if ($?) { g++
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

B.1 (ILOB)

main_bp1.cpp	

DFS.H

Welcome | main_bp1.cpp | DFS.h

DFS > DFS.h > ...

```h
#ifndef DFS_H
#define DFS_H

#include <iostream>
#include <map>
#include <set>
#include <stack>
#include <string>
#include <vector>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges) {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}
```

Welcome | main_bp1.cpp | DFS.h

DFS > DFS.h > ...

```cpp
#ifndef DFS_H
template <typename T>
class Graph {
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {}

    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }

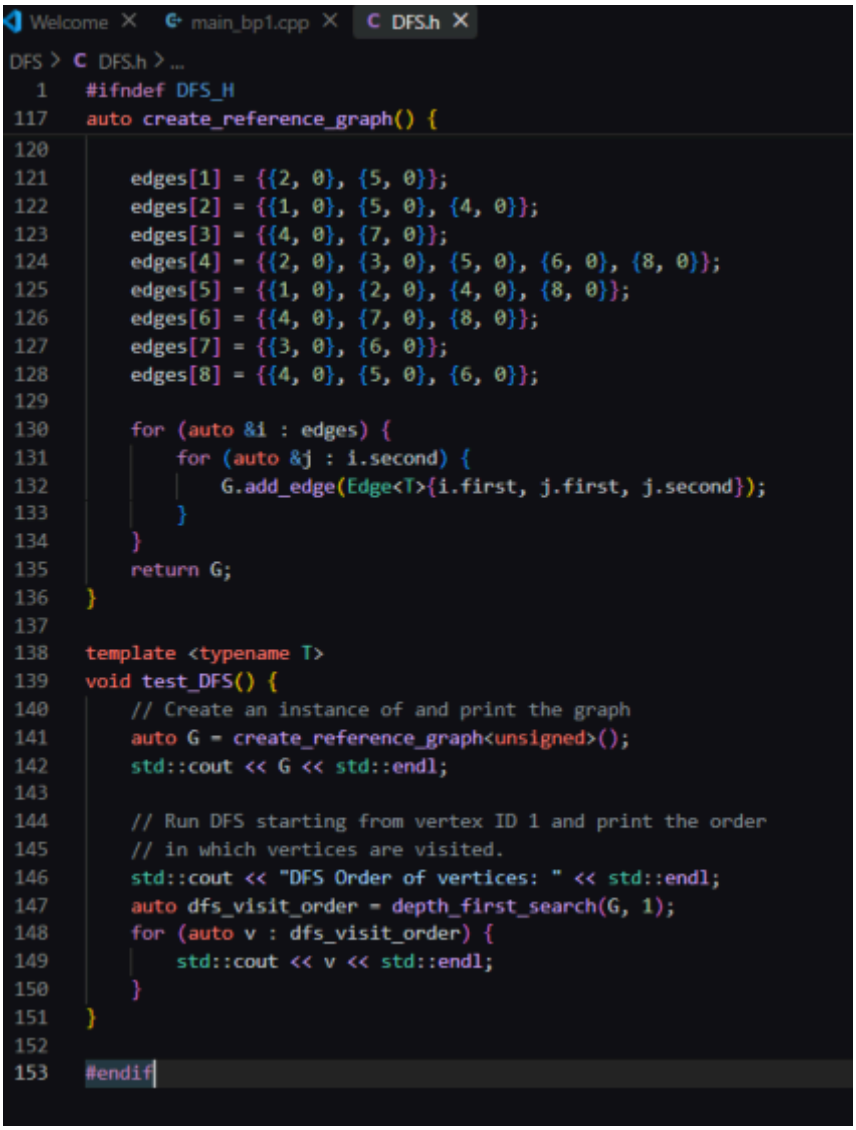
    // Return all edges in the graph
    auto &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
            edge_list.emplace_back(e);
        } else {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph can be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

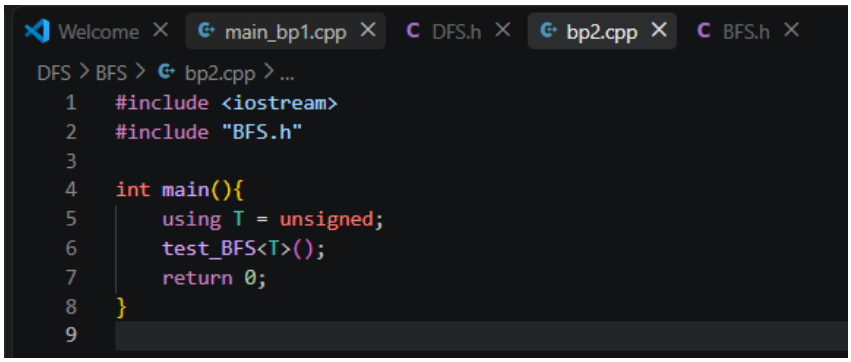
```


Welcome
main_bp1.cpp
DFS.h
DFS > DFS.h > ...
#ifndef DFS_H
auto create_reference_graph() {
edges[1] = {{2, 0}, {5, 0}};
edges[2] = {{1, 0}, {5, 0}, {4, 0}};
edges[3] = {{4, 0}, {7, 0}};
edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
edges[6] = {{4, 0}, {7, 0}, {8, 0}};
edges[7] = {{3, 0}, {6, 0}};
edges[8] = {{4, 0}, {5, 0}, {6, 0}};
for (auto &i : edges) {
for (auto &j : i.second) {
G.add_edge(Edge<T>{i.first, j.first, j.second});
}
}
return G;
}
template <typename T>
void test_DFS() {
// Create an instance of and print the graph
auto G = create_reference_graph<unsigned>();
std::cout << G << std::endl;
// Run DFS starting from vertex ID 1 and print the order
// in which vertices are visited.
std::cout << "DFS Order of vertices: " << std::endl;
auto dfs_visit_order = depth_first_search(G, 1);
for (auto v : dfs_visit_order) {
std::cout << v << std::endl;
}
}
#endif

OUTPUT


PS C:\Users\Doloso\Documents\CPE010_DOLOSO-CPE21S1> cd "c:\Users\Doloso\Documents\CPE010_DOLOSO-CPE21S1\DFS\" ; if ($
1: {2: 0}, {5: 0},
2: {1: 0}, {5: 0}, {4: 0},
3: {4: 0}, {7: 0},
4: {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5: {1: 0}, {2: 0}, {4: 0}, {8: 0},
6: {4: 0}, {7: 0}, {8: 0},
7: {3: 0}, {6: 0},
8: {4: 0}, {5: 0}, {6: 0},
DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\Doloso\Documents\CPE010_DOLOSO-CPE21S1\DFS>

BFS(ILOB2)

bp2.cpp	

BFS.h

Welcome | main_bp1.cpp | DFS.h | bp2.cpp | BFS.h

DFS > BFS > BFS.h > ...

```h
#ifndef BFS_H
#define BFS_H

#include <iostream>
#include <map>
#include <queue>
#include <set>
#include <string>
#include <vector>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges) {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const {
        return V;
    }

    auto &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
            edge_list.emplace_back(e);
        } else {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
```

Welcome | main_bp1.cpp | DFS.h | bp2.cpp | BFS.h

DFS > BFS > BFS.h > ...

```
#ifndef BFS_H
class Graph {
public:
    void add_edge(Edge<T> &&e) {
        } else {
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges) {
        for (auto &j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t start_vertex = 1) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(start_vertex);

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();

```

```
            // If the current vertex hasn't been visited in the past
            if (visited.find(current_vertex) == visited.end()) {
                visited.insert(current_vertex);
                visit_order.push_back(current_vertex);

                for (auto e : G.outgoing_edges(current_vertex)) {
                    queue.push(e.dest);
                }
            }
        }
        return visit_order;
    }

    template <typename T>
    void test_BFS() {
        // Create an instance of and print the graph
        auto G = create_reference_graph<unsigned>();
        std::cout << G << std::endl;

        // Run BFS starting from vertex ID 1 and print the order
        // in which vertices are visited.
        std::cout << "BFS Order of vertices: " << std::endl;
        auto bfs_visit_order = breadth_first_search(G, 1);
        for (auto v : bfs_visit_order) {
            std::cout << v << std::endl;
        }
    }

    #endif
```

OUTPUT

```
Problems  Output  Debug Console  Terminal  Ports

                                                    > cd "c:\Users\Doloso\Documents\CPE010_DOLOSO-CPE2151\DFS\BFS\" ; if ($?) { g++ bst.cpp
1:      [2: 2], [5: 3],
2:      [1: 3], [5: 5], [4: 1],
3:      [4: 2], [7: 3],
4:      [2: 1], [3: 2], [5: 2], [6: 4], [8: 5],
5:      [1: 3], [2: 5], [4: 3], [8: 3],
6:      [4: 4], [7: 4], [8: 3],
7:      [3: 3], [6: 4],
8:      [4: 5], [5: 3], [6: 3],

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\Doloso\Documents\CPE010_DOLOSO-CPE2151\DFS\BFS>
```

CONCLUSION:

- This hands-on-Activity helped me understand the difference between a graph and a tree. A tree is a type of non-linear data structure, while a graph is also a non-linear data structure that can have different connections between vertices. In ILO A, I learned how to create a graph using an adjacency list or edge list, add edges, and count the total number of edges. In ILO B, I learned the two types of graph traversal: Depth-First Search (DFS) and Breadth-First Search (BFS). DFS is similar to Pre-Order traversal because it follows one path first, but in a graph, there is no root node, so it starts from a chosen vertex. BFS explores the nearby vertices first before moving to the next level.

Supplementary Activity:

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- Depth-First Search (DFS) is the best algorithm for this task. DFS goes as deep as possible along each path before backtracking. This means the traveler can start at one place, Just like the Pre-order Traversal it follow connected locations step-by-step until reaching the end of that path, go back to the previous location, and then explore another route.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- In tree traversal, Pre-Order Traversal is the equivalent of DFS in graphs. Because like I said earlier, both methods visit a starting node and follow connected locations step-by-step until reaching the end of that path, go back to the previous location, and then explore another route.

PSEUDO CODE:

Pre-order Traversal :

```
START
If node is null:
Return
visit(node)
preorder(node left)
preorder(node right)
END
```

DFS :

```
START
Create stack
Push first vertex to stack
Mark first vertex as visited
While stack is not empty
Pop current vertex from stack
Visit current vertex
For each adjacent vertex of current
If adjacent vertex is not visited
Mark adjacent vertex as visited
Push adjacent vertex to stack
END
```

For the code implementation:

Pre-Order Traversal

```cpp
// Pre order
template <typename T>
void BinarySearchTree<T>::preOrder() {
    // Process current node first
    std::cout << key << " ";

    // Traverse left subtree
    if (left != nullptr) {
        left->preOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->preOrder();
    }
}
```

DFS

```cpp
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t start_vertex = 1) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(start_vertex);

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                queue.push(e.dest);
            }
        }
    }
    return visit_order;
}
```

- The main difference is that trees have a fixed root node and no cycles, while graphs have no fixed root and require tracking visited nodes to avoid getting stuck in loops.

3. In the performed code, what data structure is used to implement the Breadth First Search?
   - A Queue data structure is used to implement Breadth-First Search (BFS). It follows a First-In, First-Out approach so nodes are processed level by level in the order they are discovered.

4. How many times can a node be visited in the BFS?
   - A node is visited only once in BFS because a visited tracker keeps track of visited nodes.

Conclusion:
Provide the following:

Summary of lessons learned

- I learned how graphs work and how to traverse them using DFS and BFS. DFS uses a stack or recursion to go deep into paths, while BFS uses a queue to explore level by level.

Analysis of the procedure
- The lab procedure clearly showed how graph data structures are built and traversed in code.

Analysis of the supplementary activity
- The questions helped strengthen my understanding of real-world applications of DFS and BFS, as well as the data structures used behind them.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas
- By completing and understanding this activity. To improve further, I want to practice more on complex graph problems and algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar Correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.